Reinforcement Learning for Transfer-Function & Camera Control

Math and Implementation Reference

Kevin Kunkel
Universität Leipzig · Masterseminar: Optimierung von Transferfunktionen
im Volume Rendering mit Reinforcement Learning

September 2026

This is a short, math-focused companion to `architecture.typ`: it derives every formula the two RL sub-projects are built on, states each environment as a Markov decision process, and summarizes the learning algorithm and the hand-coded baseline both agents are compared against. Both base agents train **offline**, directly against a scalar geometric/radiometric metric computed from raw data — neither ever calls the VTK renderer. Two extensions build on the opacity agent specifically: Section 6 trains the same MDP continually instead of in one batch, and Section 7 replaces the automatic metric with a goal-conditioned reward model learned from real human preference signal mined out of ordinary app usage — the first point in this document where rendering enters the RL loop at all.

Contents

1. Shared building block: the opacity metric

Both sub-projects reuse the same transfer-function representation and the same integral metric, so it is derived once here and referenced from both.

1.1. Transfer function as a Gaussian mixture

The transfer function is a flat vector $p \in [-1, 1]^{24}$: four peaks, six parameters each — center c_i, width w_i, height h_i, and an RGB triple — always stored normalized and mapped to real units by an affine rescale $\text{to\_range}(x, \text{lo}, \text{hi}) = \text{lo} + \frac{x+1}{2} \cdot (\text{hi} - \text{lo})$. At any Hounsfield-unit value x, peak i contributes a Gaussian bump

$$g_i(x) = \exp\left(-\frac{1}{2}\left(\frac{x - c_i}{w_i}\right)^2\right), \quad \text{contrib}_i(x) = h_i \cdot g_i(x),$$

and the four peaks are composited into one clipped total opacity curve

$$\alpha(x; p) = \text{clip}\left(\sum_{i=1}^{4} \text{contrib}_i(x), 0, 1\right).$$

(Color is a contribution-weighted blend of the four peaks' RGB values, irrelevant to any RL signal and omitted here.)

1.2. `opacity_mass` and `mass_fraction`

`opacity_mass` is the exact trapezoidal integral of α over an HU sub-range $[\text{lo}, \text{hi}]$, sampled at $n = 256$ points:

$$M(p; \text{lo}, \text{hi}) = \int_{\text{lo}}^{\text{hi}} \alpha(x; p)\ \mathrm{d}x \approx \text{trapz}(\alpha(x_1; p), ..., \alpha(x_n; p)).$$

This is the ground-truth signal everything downstream is built on — no rendering needed. Five fixed HU bands (`TISSUE_BANDS`) partition the domain into tissues (air, fat, soft, spongy, bone). Because raw M scales with a band's width (bone's band alone is 1400 HU wide, vs. 520 HU for fat), the RL observation and reward instead use the width-normalized **mass fraction**

$$\varphi(p, t) = M(p; \text{lo}_t, \text{hi}_t)/(\text{hi}_t - \text{lo}_t) \in [0, 1],$$

an average-opacity-in-band figure that does not reward a policy simply for picking a wide tissue band.

2. RL sub-project 1 — transfer-function opacity

Can a learned continuous-control policy match the hand-coded hill-climber at nudging one tissue's opacity toward a goal? Formalized as an episodic MDP $(\mathcal{S}, \mathcal{A}, R)$ with deterministic transitions (`rl/env.py`):

2.1. Episode setup

On `reset()`, all four peaks' heights are perturbed by uniform noise ± 0.3 from `default_params()`, then a target tissue $t \in \{\text{air}, \text{fat}, \text{soft}, \text{spongy}, \text{bone}\}$ and a goal direction $d \in \{+1, -1\}$ (increase/decrease) are sampled uniformly. The target peak index is resolved once via the same `_find_or_create_peak` the command layer uses, so the agent moves the vector through the identical mechanism a real user command would.

2.2. State, action, reward

$$s = (p, \text{onehot}(t), d, \varphi(p, t)) \in \mathbb{R}^{24+5+1+1} = \mathbb{R}^{31}$$

$$a \in [-1, 1], \quad \Delta = a \cdot 0.2$$

The action scales a single scalar delta ($\text{MAX\_DELTA} = 0.2$) that is added directly to the target peak's **unit** height (the height slot converted from $[-1, 1]$ to $[0, 1]$, clipped back to $[0, 1]$) — no direction sign is baked into the transition; the agent must learn to pick the sign of a itself from d in the observation:

$$h^{\text{unit}}_{\text{new}} = \text{clip}(h^{\text{unit}}_{\text{old}} + \Delta, 0, 1).$$

The reward is the signed per-step change in mass fraction toward the goal:

$$r = \text{sign}(d) \cdot (\varphi(p', t) - \varphi(p, t)), \quad \text{sign}(d) = \begin{cases} +1 & d = \text{increase} \\ -1 & d = \text{decrease} \end{cases}.$$

Episodes truncate (not terminate) after 20 steps — there is no terminal state, only a fixed interaction budget, matching how a real editing session has no natural "done."

2.3. Result (200k timesteps, SAC, 20 held-out episodes)

	Mean final φ	**Mean steps to 90% of best**
Trained SAC policy	0.344	3.3
Hill-climb baseline	0.347	2.7

Near-parity: on this low-dimensional, single-scalar-action problem, a well-tuned hand rule is a strong baseline. Not wired into the live chat/voice loop.

3. RL sub-project 2 — camera viewpoint

Orient the camera toward a target tissue's centroid direction, against a cheap exact geometric proxy computed directly from the volume's voxel data — no rendering, no learned reward model (`rl/camera_env.py`).

3.1. View direction and target direction

The camera is parameterized by azimuth/elevation angles; its unit view direction in the volume's coordinate frame is

$$v(\mathrm{az}, \mathrm{el}) = \begin{pmatrix} \cos(\mathrm{el})\sin(\mathrm{az}) \\ \sin(\mathrm{el}) \\ \cos(\mathrm{el})\cos(\mathrm{az}) \end{pmatrix}.$$

A target tissue's direction is computed once per episode directly from the HU volume: mask the voxels in the tissue's band, take their physical-space centroid, and normalize its offset from the volume's geometric center:

$$\bar{c}_t = \text{voxel-mean}(\{x : \mathrm{lo}_t \le \mathrm{HU}(x) < \mathrm{hi}_t\}) \cdot \text{spacing}, \quad \hat{d}_t = \frac{\bar{c}_t - c_{\text{vol}}}{\|\bar{c}_t - c_{\text{vol}}\|}.$$

A candidate direction is rejected (falls back to the next tissue, ultimately `bone`) if its offset is below 2% of the volume's diagonal extent — too close to the center to have a meaningful direction, distinguishing real anatomical asymmetry from grid-discretization noise.

3.2. State, action, reward

$$s = (\sin(\mathrm{az}), \cos(\mathrm{az}), \mathrm{el}/85, \mathrm{onehot}(t), A) \in \mathbb{R}^9, \quad a \in [-1, 1]^2$$

$$\mathrm{az}' = (\mathrm{az} + a_1 \cdot 30°) \bmod 360°, \quad \mathrm{el}' = \mathrm{clip}(\mathrm{el} + a_2 \cdot 30°, -85°, 85°)$$

Alignment is the cosine similarity between the current view direction and the fixed target direction (both unit vectors, so a plain dot product):

$$A(\mathrm{az}, \mathrm{el}) = v(\mathrm{az}, \mathrm{el}) \cdot \hat{d}_t \in [-1, 1], \qquad r = A' - A.$$

Azimuth **wraps** (no natural bound); elevation **clamps** (avoids flipping through the poles) — the same asymmetry the manual camera commands use. 20-step truncated episodes, as in Section 2.

> **A real bug caught during this sub-project:** The hill-climb baseline seeds its step at 30°, but the shared `resize_step` growth cap defaulted to 1.0 — correct for opacity values in $[0, 1]$, wrong by 30× here. The very first accepted move collapsed the step from 30° to 1° with no way to grow back: mean final alignment **0.658**. Adding an explicit `max_step` parameter (default 1.0, so the three opacity-scale callers are unaffected) and passing `max_step=30` at this call site fixed it: **0.99998**.

3.3. Result (200k timesteps, SAC, `ct_skull`, 20 held-out episodes)

	Mean final alignment	Mean steps to 90% of best
Trained SAC policy	0.997	3.05
Hill-climb baseline	1.000	4.05

Both reach near-perfect alignment — the alignment landscape is a smooth, unimodal dot product against a fixed direction, easy for either method. The trained policy's edge is convergence speed (fewer renders needed in an interactive tool), not final quality. The reward has **no occlusion model** — it is a directional proxy for "facing the tissue," not "can see it unobstructed."

4. Learning algorithm: Soft Actor-Critic

Both agents are trained with `stable-baselines3`'s SAC (`MlpPolicy`, default two-hidden-layer-of-256 network, 4 parallel envs via `make_vec_env`, `gamma = 0.99` default). SAC is an off-policy, maximum-entropy actor-critic method: alongside expected return, the policy π is trained to maximize its own entropy $\mathcal{H}$, trading a bit of reward for exploration robustness —

$$J(\pi) = \mathbb{E}_{\pi}\left[\sum_{t} \gamma^{t}(r_t + \alpha_{\text{ent}}\mathcal{H}(\pi(\cdot\,|s_t)))\right],$$

with twin clipped Q-networks (reducing value overestimation) and an automatically-tuned entropy coefficient α_{ent} (both SB3 defaults, not overridden here). Both environments are pure Python/NumPy with no rendering in the loop, so training throughput is bounded by CPU, not GPU or I/O.

5. Baseline: hill climbing (`search.py`)

The comparison point for both sub-projects, and the system's actual production optimizer:

$$\text{propose\_step}(p, i, s, \delta): \quad h_i^{\text{unit}} \leftarrow \text{clip}(h_i^{\text{unit}} + s \cdot \delta, 0, 1)$$

$$\text{resize\_step}(\delta, \text{accepted}, \delta_{\max}) = \begin{cases} \min(1.2\delta, \delta_{\max}) & \text{accepted} \\ 0.5\delta & \text{rejected} \end{cases}$$

— grow the step $1.2\times$ on acceptance (capped at a domain-appropriate $\delta_{\max}$), halve it on rejection. Acceptance for a relative increase/decrease command is decided by `evaluate.objective`: let $\Delta_t = \pm(M(p', \text{target}) - M(p, \text{target}))$ (sign matching the requested direction). The step is accepted iff it actually moved the target band the right way **and** no other tissue's mass moved by more — a collateral-damage check with no RL analogue:

$$\text{accept} \Leftrightarrow \Delta_t > \varepsilon \wedge \max_{t' \neq t} |M(p', t') - M(p, t')| \leq \Delta_t, \quad \varepsilon = 10^{-4}.$$

Both `rl/eval.py` and `rl/camera_eval.py` measure **steps to 90% of best** generically: given the trajectory's start value f_0 and the best value either method reached across the pair, the step index at which f first crosses $f_0 + 0.9(f_{\text{best}} - f_0)$ (or the symmetric case for a decreasing goal).

> **Scope:** Both base agents are trained and evaluated entirely offline against the metrics above. The camera policy is not reachable from the live chat/voice loop; the opacity policy now is, as frozen inference (`server.py`'s `policy` search toggle, via `rl/serve.py`) — see Section 6 and Section 7 below for the two ways training itself is being extended. See `architecture.typ` for the full system this feeds into.

6. Extension: continual online learning

`rl/train.py` trains once, offline, in a single 200k-step batch across 4 parallel environments, evaluated once at the end. `rl/online_train.py` trains the **identical** MDP from Section 2 — same state, action, reward — but as one continuous stream on a single environment, checkpointing held-out performance periodically instead of only at the end, so the metric's improvement over training is directly observable.

6.1. Mechanics

Training proceeds in chunks of `eval_interval` steps. Each chunk is a plain SB3 call — no custom replay-buffer or gradient-step code — so the online variant's **learning** mechanics are identical to the offline agent's; only the scheduling around it differs:

$$\text{for } k = 1, \ldots, N/E: \quad \text{model.learn}(\text{total\_timesteps} : E, \text{reset\_num\_timesteps} : \text{False})$$

where N is the total step budget and E the eval interval. After each chunk, the current policy is replayed (deterministically) against the same 20 held-out episodes Section 2's offline evaluation uses, and the mean final φ and mean steps-to-90% are logged — the exact same two statistics the offline result table reports, making the two directly comparable.

6.2. Result ($N = 50\text{k}$ steps, $E = 5\text{k}$, single env)

Steps	Mean final φ	Mean steps to 90%
5,000	0.199	17.35
10,000	0.281	16.00
15,000	0.347	9.35
20,000	0.347	5.15
25,000	0.346	3.80
30,000	0.346	3.30
35,000	0.344	3.30
40,000	0.345	3.25
45,000	0.346	3.20
50,000	0.345	3.20

Note: 50k single-env steps are **25%** of the offline agent's 200k-step budget (SB3's `total_timesteps` already counts across all parallel envs, so this is a direct comparison, not one that needs a $\times 4$ correction). The online run lands at 0.345 — almost exactly between the offline policy (0.344) and hill-climb baseline (0.347) — while steps-to-90%-of-best falls steadily from 17.4 to 3.2: a clean, converging curve on a quarter of the budget, not a coincidental match.

7. Extension: goal-conditioned RLHF reward model

φ (Section 2) measures only whether the TF curve moved the way a command literally asked — never whether the resulting **image** looks good. This extension trains a reward model from real human preference signal, then trains RL against that learned reward instead of φ's exact delta — the RLHF pattern, and the first point in this document where the VTK renderer enters the RL loop at all. Unlike a generic image-quality score, the model is **goal-conditioned**: it sees which tissue and direction the user actually asked for, so it learns whether a transition served that specific request, not just whether the resulting frame looks nicer in general.

7.1. Preference sources and extraction

No dedicated labeling session is required. `rl/extract_pairs.py` mines three signals the application already produces during ordinary use, each trusted in proportion to how unambiguous it is:

Source	Signal	Weight w
Branch	Explicit A/B: two children of the same parent step, one accepted	1.0
Trajectory	Within an episode, an accepted/ended step vs. an earlier one ($k - j \geq 2$)	0.7
Thumbs	Standalone up/down rating, joined to a rendered step by session/step	0.4

Branches are extracted only from explicit parent/carried-forward metadata — never inferred from similar parameters — so historical logs that predate branch tracking correctly report zero branch pairs rather than guessing. Each extracted pair carries its source weight w_i through to training.

7.2. Goal-conditioned featurization

`render.features(rgb)` reduces a rendered frame to four cheap scalars:

$$f(x) = (\text{mean}(x), \text{std}(x), \text{coverage}(x), \text{entropy}(x)) \in \mathbb{R}^4.$$

Where the design changes is the observation itself. Each side of a pair is encoded as **after**, **before**, their **delta**, and the goal that was actually asked for — an 18-dimensional vector instead of the earlier 10-dimensional, goal-blind one:

$$z = (f(\text{after}), f(\text{before}), f(\text{after}) - f(\text{before}), \text{onehot}(t), d) \in \mathbb{R}^{4+4+4+5+1} = \mathbb{R}^{18},$$

with t the target tissue (one of the five tissue classes) and $d \in \{+1, -1\}$ the requested direction (increase/decrease).

7.3. Reward model and preference loss

A small MLP $R_\theta : \mathbb{R}^{18} \to \mathbb{R}$ ($18 \to 64 \to 32 \to 1$, ReLU) scores a single observation. Training operates on **pairs**, not single labels: given a preferred observation z_i^+ and the alternative z_i^-, the weighted Bradley-Terry loss pushes the preferred score higher in proportion to how confident that pair's source is:

$$\mathcal{L}(\theta) = -\frac{\sum_i w_i \log \sigma(R_\theta(z_i^+) - R_\theta(z_i^-))}{\sum_i w_i}.$$

At inference the reward handed to RL is, as before, the sigmoid output remapped to a signed range:

$$\tilde{r}(z) = 2\sigma(R_\theta(z)) - 1 \in [-1, 1].$$

7.4. Two-phase training: synthetic pretraining, then human fine-tuning

Real preference data is scarce by construction — a handful of pairs from ordinary usage, not a curated dataset — so the model never trains from a random initialization on that data alone.

Pretraining (`rl/pretrain_reward.py`) generates a configurable number of **synthetic** pairs (default 20k): a random starting TF vector, a random goal (t, d), and two random deltas, both rendered and labeled by which one moved the **signed** `mass_fraction` objective further — the same φ from Section 2, not a human label. This only warm-starts the weights into a sane region; an in-code `TODO` marks the exact spot where a genuine rendered-visibility metric should eventually replace this proxy. An ensemble of $K = 5$ independently seeded members is trained this way by default (see the reward composition below for why).

Fine-tuning (`rl/finetune_reward.py`) loads the pretrained ensemble and continues training — **only** on real extracted pairs — at one tenth the pretraining learning rate, on a fixed, seeded train/test split that is persisted in the checkpoint metadata and never touched again by later evaluation runs. Pretrained and fine-tuned checkpoints are saved as separate artifact families; fine-tuning never overwrites the cold-start weights, so both can always be compared against each other.

7.5. Reward composition and anti-hacking safeguards

`RewardModelTFEnv` subclasses `TFEnv` from Section 2 unchanged — same sampling, same action, same observation shape (still including the free φ value) — and overrides only the reward. As before, it caches the previous step's rendered features so a rollout of k steps costs $k + 1$ renders, not $2k$.

An earlier version of this document measured **109 ms per render** on the synthetic phantom and used that to justify keeping this extension's step budgets two orders of magnitude below the automatic-metric agents'. That number was almost entirely native offscreen-GL-context creation cost, not raycast cost: `render.render()` built a brand-new `vtkRenderWindow` on every call, and any loop rendering hundreds of times in one process (this extension's training loop, and `rl/pretrain_reward.py`'s synthetic-pair generation) reliably hung after roughly a minute of wall-clock time, independent of total workload size — root-caused with `sample` on the hung process down to a macOS WindowServer/IOKit surface-bind call (`IOAccelCreateSurface`) that stops responding after enough repeated context-creation requests from one non-windowed process. `render.py` now caches and reuses the render pipeline per volume instead of rebuilding it every call, which removes the hang entirely and brings steady-state cost down to **≈ 7 ms per render** (measured: 400 renders in 2.9 s with one reused window) — freeing this extension's step budgets from the render-cost constraint that originally motivated them.

The reward itself is no longer the ensemble's raw output. Given K member scores $R_{\theta_1}(z), \ldots, R_{\theta_K}(z)$ (each already remapped via $\tilde{r}$), the **ensemble reward** is mean penalized by disagreement:

$$r_{\text{model}(z)} = \bar{\tilde{r}}(z) - s(z), \quad \bar{\tilde{r}}(z) = \frac{1}{K}\sum_{k=1}^{K} \tilde{r}_k(z), \quad s(z) = \sqrt{\frac{1}{K}\sum_{k=1}^{K} \left(\tilde{r}_k(z) - \bar{\tilde{r}}(z)\right)^2}.$$

States where the ensemble disagrees — plausibly outside the training distribution — are automatically discounted, without any explicit out-of-distribution detector. The final reward blends this against the untouched objective anchor:

$$r = \alpha \cdot r_{\text{model}(z)} + (1 - \alpha) \cdot \varphi_{\text{signed}}, \quad \alpha \in [0, 1],$$

with default $\alpha = 0.7$; $\alpha = 0$ recovers the pre-RLHF automatic-metric agent exactly, and $\alpha = 1$ trains against the learned reward alone — both are one-line ablations, not separate code paths. Independently of α, a **hard** penalty $r = -1.0$ overrides everything whenever a state is degenerate — `coverage` below 0.01 (near-black) or mean opacity above 0.95 (fully opaque) — so the policy cannot win by rendering something the learned model happens to score well but no human would call useful.

7.6. Evaluation: three independent checks

`rl/eval_reward.py` keeps three questions separate, none of them feeding back into training:

1. **Reward accuracy** — pretrained vs.
   fine-tuned agreement with human labels on the same held-out split, reported overall and per source, so a single branch pair and ten low-confidence thumbs pairs are not silently averaged together.

2. **Policy sanity** — the RLHF-trained policy's performance on the plain automatic metric, as a collapse/reward-hacking check independent of the learned reward that trained it.
3. **Blind head-to-head** — the RLHF policy vs. the hill-climb baseline, randomized A/B order, judged by a human with no visibility into which policy produced which image; verdicts are appended only to a separate results file.

7.7. Growing the real preference volume

Extraction alone does not create signal that was never recorded — it only recovers what is already there. Three ways to add real labeled pairs, weakest effort first:

1. **Ordinary chat-UI usage** — thumbs up/down while using `server.py` normally. Zero extra effort; joined to rendered steps as `thumbs` pairs (weight 0.4) by `rl/extract_pairs.py`'s default `--feedback` path.
2. **Human-judged hill-climbing** — `python mvp.py --cmd "..." --learn --human --steps 10` logs every judged step to `out/preferences.jsonl` via `log_preference()` — a **third**, separate file from `extract_pairs`'s three defaults (`out/log.jsonl`, `out/feedback.jsonl`, `out/rlhf_preferences.jsonl`). It must be pointed at explicitly — `rl/extract_pairs.py --preferences out/preferences.jsonl` — or these labels are silently never extracted.
3. **Dedicated labeling** — `python -m rl.collect_preferences --n-pairs 50 --rater-id <name>` gets a batch of `thumbs`-weighted pairs in one sitting instead of accumulating incidentally.

None of these raises a pair's **source weight** — a dedicated session still produces 0.4-weight `thumbs` pairs, same as passive usage; only explicit branch comparisons reach 1.0. Growing n is what turns fine-tuned accuracy from a plumbing check into an actual result.

> **Scope:** End-to-end mechanics are validated against real data — extraction, pretraining, fine-tuning, evaluation, and RL training against the fine-tuned model all run cleanly on real logs, and two real bugs surfaced this way (not by the test suite) are fixed: preference rows embedded in `log.jsonl`/`feedback.jsonl` were previously invisible to the extractor, and multi-target commands ("show only bone and fat") crashed it outright. What remains a pilot is **volume**: real usage has produced only a handful of extractable pairs so far, so fine-tuned accuracy numbers are not yet a generalizable result and are reported honestly as such, per the design's explicit methodological stance — a real 2000-step RL training run against this pilot-scale fine-tuned model shows an essentially flat learned-reward score across training, the expected signature of too little data, not a training bug. Camera-viewpoint RLHF is out of scope for now.